
AutoGrade — Hệ thống chấm điểm code tự động

Báo cáo đặc tả phi chức năng

Đồ án môn Công nghệ phần mềm cho AI

Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM (HCMUS)

1. Giới thiệu

1.1 Mục đích tài liệu

Tài liệu này đặc tả **yêu cầu phi chức năng (Non-Functional Requirements — NFR)** của hệ thống AutoGrade — nền tảng chấm điểm code tự động. Trong khi bản *Đặc tả Use-Case* mô tả hệ thống **làm gì** (functional requirements), tài liệu này mô tả hệ thống **làm tốt tới mức nào** — tức các thuộc tính chất lượng như bảo mật, hiệu năng, độ tin cậy, khả chuyển, khả dụng và khả bảo trì. Cả hai cùng cấu thành phần Software Requirements theo chuẩn RUP/SRS.

1.2 Phạm vi

Các NFR ở đây được rút ra và kiểm chứng trực tiếp từ mã nguồn hiện thực của AutoGrade (backend Node.js/Express/TypeScript, frontend React/Vite/TypeScript, cơ sở dữ liệu PostgreSQL trên Supabase) cùng năm mô-đun của pipeline chấm điểm. Mỗi yêu cầu đều kèm bằng chứng là vị trí trong mã nguồn, nên đây là đặc tả “as-built” (mô tả đúng những gì đã xây), thay vì một danh sách kỳ vọng lý thuyết.

1.3 Khung phân loại

Tài liệu tổ chức NFR theo bảy nhóm thuộc tính chất lượng của **ISO/IEC 25010 (Product Quality Model)** — chuẩn quốc tế phổ biến nhất cho chất lượng phần mềm. Bảy nhóm được chọn vì phù hợp với đặc thù AutoGrade (chạy code không tin cậy, gọi dịch vụ AI ngoài, xử lý dữ liệu học vụ):

Nhóm chất lượng (ISO/IEC 25010)	Mã	Nội dung trọng tâm với AutoGrade
Bảo mật (Security)	SEC	Xác thực, phân quyền, băm mật khẩu, cách ly code, chống dò email.
Hiệu năng (Performance Efficiency)	PER	Timeout sandbox/LLM, xử lý song song, tính tắt định của điểm.
Độ tin cậy (Reliability)	REL	Suy giảm mềm khi AI lỗi, retry, theo dõi job, health-check.
Tính khả chuyển (Portability)	POR	Đa nền tảng OS, đa ngôn ngữ lập trình, độc lập hạ tầng.
Tính khả dụng (Usability)	USA	Bản địa hoá tiếng Việt, chạy thử trước khi nộp, thông báo lỗi rõ.
Khả bảo trì (Maintainability)	MNT	Kiến trúc mô-đun, TypeScript, tách lớp, self-test.
Khả mở rộng (Scalability)	SCL	Giới hạn hiện tại & hướng nâng cấp lên hàng đợi/Docker.

1.4 Tổng quan kiến trúc (bối cảnh của NFR)

AutoGrade dùng kiến trúc tách biệt frontend/backend, giao tiếp qua REST API (JSON + JWT). Bối cảnh này quyết định phần lớn các NFR bên dưới:

- **Frontend:** React 18 + Vite + TypeScript; editor code dùng Monaco; xuất Excel phía client bằng thư viện `xlsx`.
- **Backend:** Node.js (≥ 18) + Express + TypeScript; là nơi duy nhất chạm cơ sở dữ liệu.
- **Cơ sở dữ liệu:** PostgreSQL trên Supabase, truy cập bằng `service_role` key ở phía server.
- **Dịch vụ AI ngoài:** Google Gemini (chấm phong cách/gợi ý sửa lỗi) và Groq (viết nhận xét tổng hợp).
- **Sandbox thực thi:** tiến trình con biên dịch/chạy code Python, C++, Java với giới hạn thời gian và bộ nhớ.

2. Bảng tóm tắt yêu cầu phi chức năng

Bảng dưới liệt kê toàn bộ NFR kèm mức ưu tiên. Phần 3 đặc tả chi tiết từng yêu cầu. (Ưu tiên: Bắt buộc = cốt lõi cho đồ án; Nên có = tăng chất lượng; Tương lai = ngoài phạm vi bản hiện tại.)

ID	Yêu cầu (tóm tắt)	Nhóm	Ưu tiên
NFR-SEC-01	Băm mật khẩu bằng bcrypt, không lưu plaintext	SEC	Bắt buộc
NFR-SEC-02	Xác thực bằng JWT có thời hạn; phân quyền theo vai trò	SEC	Bắt buộc
NFR-SEC-03	Bắt buộc xác nhận email mới cho đăng nhập	SEC	Bắt buộc
NFR-SEC-04	Chống dò email (phản hồi chung, token hết hạn)	SEC	Nên có
NFR-SEC-05	Cách ly khoá bí mật; service_role chỉ ở backend	SEC	Bắt buộc
NFR-SEC-06	Cách ly & giới hạn tài nguyên khi chạy code người dùng	SEC	Bắt buộc
NFR-SEC-07	Chống gian lận học thuật (cổng phát hiện đạo văn)	SEC	Bắt buộc
NFR-SEC-08	Giới hạn kích thước dữ liệu vào (body/file)	SEC	Nên có
NFR-PER-01	Timeout biên dịch/chạy code trong sandbox	PER	Bắt buộc
NFR-PER-02	Timeout + retry có backoff khi gọi LLM	PER	Bắt buộc
NFR-PER-03	Chấm điểm tắt định, tái lập được	PER	Bắt buộc
NFR-PER-04	Chạy song song mô-đun độc lập; biên dịch một lần	PER	Nên có
NFR-REL-01	Suy giảm mềm khi thiếu khoá AI / AI lỗi	REL	Bắt buộc
NFR-REL-02	Theo dõi tiến trình chấm qua bảng grading_jobs	REL	Nên có
NFR-REL-03	Lưu file bền vững, có phương án dự phòng	REL	Nên có
NFR-REL-04	Health-check; lỗi chấm không chặn việc nộp	REL	Nên có
NFR-POR-01	Chạy đa nền tảng Windows/macOS/Linux	POR	Bắt buộc
NFR-POR-02	Hỗ trợ nhiều ngôn ngữ lập trình (Python/C++/Java)	POR	Bắt buộc
NFR-POR-03	Cấu hình qua biến môi trường; độc lập hạ tầng	POR	Nên có
NFR-USA-01	Bản địa hoá tiếng Việt toàn hệ thống	USA	Bắt buộc
NFR-USA-02	Chạy thử trước khi nộp, không tính điểm	USA	Nên có
NFR-USA-03	Thông báo lỗi rõ ràng, có hướng dẫn khắc phục	USA	Nên có
NFR-MNT-01	Toàn bộ TypeScript kiểu chặt; có typecheck	MNT	Bắt buộc
NFR-MNT-02	Kiến trúc mô-đun, tách lớp route/service/grading	MNT	Bắt buộc
NFR-MNT-03	Script self-test cho pipeline chấm điểm	MNT	Nên có
NFR-SCL-01	Kiến trúc stateless cho phép mở rộng ngang	SCL	Nên có
NFR-SCL-02	Nâng cấp sang hàng đợi & sandbox container	SCL	Tương lai

3. Đặc tả chi tiết theo nhóm chất lượng

3.1 Bảo mật (Security)

Vì AutoGrade lưu dữ liệu học vụ (điểm, bài nộp) và đặc biệt là thực thi code do người dùng gửi lên, bảo mật là nhóm NFR quan trọng nhất. Hệ thống áp dụng phòng thủ nhiều lớp: xác thực, phân quyền, bảo vệ tài khoản, cách ly thực thi và liêm chính học thuật.

NFR-SEC-01 — Băm mật khẩu

Phát biểu yêu cầu	Mật khẩu người dùng phải được băm một chiều bằng bcrypt (salt tự sinh) trước khi lưu; hệ thống không bao giờ lưu hay ghi log mật khẩu dạng plaintext.
Lý do (Rationale)	Nếu cơ sở dữ liệu bị lộ, bcrypt (có salt và cost factor) khiến việc dò ngược mật khẩu trở nên bất khả thi trong thực tế, bảo vệ tài khoản sinh viên và giảng viên.
Bằng chứng trong mã nguồn	routes/auth.ts: <code>bcrypt.hash(password, 10)</code> khi đăng ký/đặt lại mật khẩu routes/auth.ts: <code>bcrypt.compare(...)</code> khi đăng nhập db/schema.sql: cột <code>users.password_hash</code> text not null
Use-case liên quan	UC Đăng ký tài khoản (2.1), UC Đăng nhập (2.3), UC Quên & đặt lại mật khẩu (2.4).
Cách kiểm chứng	Kiểm tra bản ghi users trong DB: trường password_hash có tiền tố "\$2" của bcrypt, độ dài 60 ký tự; không có cột lưu plaintext.
Ưu tiên	Bắt buộc

NFR-SEC-02 — Xác thực JWT & phân quyền theo vai trò

Phát biểu yêu cầu	Sau đăng nhập, hệ thống cấp JWT ký bằng khoá bí mật, chứa id và role, có thời hạn (mặc định 7 ngày). Mọi API nghiệp vụ yêu cầu Bearer token hợp lệ; các thao tác của giảng viên yêu cầu đúng vai trò lecturer.
Lý do (Rationale)	JWT stateless giúp backend xác thực mà không cần lưu phiên, đồng thời phân tách rõ quyền của sinh viên và giảng viên (ví dụ chỉ chủ lớp mới tạo bài/chấm/công bố điểm).
Bằng chứng trong mã nguồn	lib/tokens.ts: <code>signJwt()</code> ký {sub, role}, expiresIn = JWT_EXPIRES_IN (mặc định 7d) middleware/auth.ts: <code>requireAuth()</code> đọc Bearer, gán req.user; <code>requireRole('lecturer')</code> index.ts: các router gán middleware trước handler
Use-case liên quan	UC Đăng nhập (2.3) là cổng xác thực; UC Tạo lớp/Tạo bài/Chấm/Công bố điểm đòi vai trò giảng viên.
Cách kiểm chứng	Gọi API nghiệp vụ không kèm token → 401; gọi API giảng viên bằng token sinh viên → 403; token hết hạn → 401.
Ưu tiên	Bắt buộc

NFR-SEC-03 — Bắt buộc xác nhận email

Phát biểu yêu cầu	Tài khoản mới ở trạng thái email_verified = false và không thể đăng nhập cho tới khi nhập đúng mã xác nhận 6 số gửi qua email.
Lý do (Rationale)	Ngăn đăng ký bằng email không có thật/không sở hữu, đảm bảo mỗi tài khoản gắn với một hộp thư kiểm soát được.
Bằng chứng trong mã nguồn	routes/auth.ts: login trả 403 "Email chưa được xác nhận" nếu !email_verified

	lib/tokens.ts: randomCode() sinh mã 6 chữ số db/schema.sql: users.email_verified boolean default false, verify_token text
Use-case liên quan	UC Xác nhận email (2.2); Alternative flow “Email chưa xác nhận” của UC Đăng nhập (2.3).
Cách kiểm chứng	Đăng ký rồi đăng nhập ngay khi chưa xác nhận → nhận 403; sau khi nhập đúng mã → đăng nhập thành công.
Ưu tiên	Bắt buộc

NFR-SEC-04 — Chống dò email (account enumeration)

Phát biểu yêu cầu	Các luồng “quên mật khẩu” và “gửi lại mã xác nhận” luôn trả về một thông báo chung, không tiết lộ email có tồn tại hay không; mã đặt lại mật khẩu hết hạn sau 1 giờ.
Lý do (Rationale)	Nếu phản hồi khác nhau giữa email tồn tại và không tồn tại, kẻ tấn công có thể dò ra danh sách email hợp lệ. Token có thời hạn giảm cửa sổ bị lạm dụng.
Bằng chứng trong mã nguồn	routes/auth.ts (forgot-password, resend-verification): trả message giống nhau bất kể email có tồn tại routes/auth.ts: reset_expires kiểm tra hết hạn; token đặt lại sống 1 giờ README.md mục 5.3: “API luôn trả thông báo giống nhau...”
Use-case liên quan	UC Quên & đặt lại mật khẩu (2.4); Alt flow “Gửi lại mã xác nhận” của UC Xác nhận email (2.2).
Cách kiểm chứng	Gửi yêu cầu quên mật khẩu với email không tồn tại và email tồn tại → hai phản hồi giống hệt; dùng mã đặt lại sau 1 giờ → bị từ chối.
Ưu tiên	Nên có

NFR-SEC-05 — Cách ly khoá bí mật

Phát biểu yêu cầu	Mọi bí mật (JWT_SECRET, Supabase service_role key, khoá API Gemini/Groq, SMTP) chỉ tồn tại ở backend qua biến môi trường; không khoá nào được nhúng vào frontend hay gửi ra client.
Lý do (Rationale)	service_role bỏ qua RLS và có toàn quyền trên DB; nếu lộ ra frontend, bất kỳ ai cũng có thể đọc/sửa toàn bộ dữ liệu. Đặt bí mật ở server là ranh giới tin cậy cốt lõi.
Bằng chứng trong mã nguồn	config.ts: đọc toàn bộ khoá từ process.env; không export ra client README.md: “service_role... chỉ đặt ở backend, không bao giờ đưa ra frontend” Frontend chỉ giữ JWT do backend cấp (lib/api.ts, localStorage)
Use-case liên quan	Xuyên suốt; nền tảng cho mọi UC gọi DB và dịch vụ AI.
Cách kiểm chứng	Rà build frontend: không tìm thấy chuỗi service_role/JWT_SECRET/API key; mọi truy cập DB đều qua Express.
Ưu tiên	Bắt buộc

NFR-SEC-06 — Cách ly & giới hạn tài nguyên khi chạy code người dùng

Phát biểu yêu cầu	Code do sinh viên nộp phải được biên dịch/chạy trong tiến trình con tách biệt, trong thư mục tạm dùng-một-lần, với giới hạn thời gian chạy (mặc định 5s), thời gian biên dịch (15s), bộ nhớ (256MB) và kích thước output (1MB).
Lý do (Rationale)	Thực thi code không tin cậy là bề mặt tấn công nguy hiểm nhất. Timeout chặn vòng lặp vô hạn/TLE, ulimit chặn ngốn RAM, thư mục tạm cô lập tệp — bảo vệ máy chủ chấm khỏi bị treo hoặc lạm dụng.
Bằng chứng trong mã nguồn	config.sandbox: runTimeoutMs=5000, compileTimeoutMs=15000, memoryMb=256, maxOutputBytes=1_000_000 grading/sandbox.ts: mkdtempSync tạo thư mục tạm, cleanup() xoá sau khi xong grading/sandbox.ts: ulimit -v (Linux/macOS), -Xmx (Java); SIGKILL khi quá giờ
Use-case liên quan	UC Chấm điểm tự động (2.9); UC con Chạy testcase (sandbox); UC Chạy thử code.
Cách kiểm chứng	Nộp code vòng lặp vô hạn → bị kill sau ~5s (TLE), không treo server; nộp code cấp phát RAM lớn → bị giới hạn.
Ưu tiên	Bắt buộc

Lưu ý giới hạn: Nhóm mã nguồn tự nhận đây là sandbox “nhẹ” (tiến trình + timeout + ulimit), đủ cho môi trường đồ án/tin cậy. Khi triển khai thật cần chạy trong Docker/nsjail để cách ly hệ thống tệp và mạng triệt để hơn — xem NFR-SCL-02.

NFR-SEC-07 — Liêm chính học thuật (cổng chống đạo văn)

Phát biểu yêu cầu	Trước khi chấm điểm bài lớp, hệ thống so khớp độ giống giữa các bài nộp; bài vượt ngưỡng rubric bị đánh 0 điểm và dừng pipeline (cổng chặn), kèm chỉ ra bài trùng cao nhất.
Lý do (Rationale)	Chấm điểm mà bỏ qua đạo văn sẽ khuyến khích sao chép. Phương pháp k-gram + winnowing bền với việc đổi tên biến, format lại, thêm/bớt comment — khó lách hơn so khớp văn bản thô.
Bằng chứng trong mã nguồn	grading/plagiarism.ts: tokenize chuẩn hoá + fingerprint winnowing (K=5, W=4), đo containment grading/pipeline.ts: GATE chạy trước; similarityPct ≥ threshold → cheatingResult(), 0 điểm, dừng Bỏ qua ở chế độ practice (bài tự luyện không có “lớp” để so)
Use-case liên quan	UC Chấm điểm tự động (2.9): «include» Phát hiện gian lận (GATE).
Cách kiểm chứng	Nộp hai bài gần như trùng → bài bị đánh dấu gian lận, 0 điểm, có tham chiếu bài trùng; bài tự luyện không bị kiểm tra.
Ưu tiên	Bắt buộc

NFR-SEC-08 — Giới hạn kích thước dữ liệu vào

Phát biểu yêu cầu	Body JSON tối đa 2MB; file upload tối đa 10MB.
Lý do (Rationale)	Chặn tấn công làm cạn bộ nhớ bằng payload khổng lồ, đồng thời giữ tài nguyên máy chủ ổn định.
Bằng chứng trong mã nguồn	index.ts: express.json({ limit: "2mb" }) lib/upload.ts: multer limits.fileSize = 10 * 1024 * 1024
Use-case liên quan	UC Nộp bài tập (2.8): «extend» Nộp kèm file.
Cách kiểm chứng	Gửi body > 2MB hoặc file > 10MB → bị từ chối trước khi xử lý.
Ưu tiên	Nên có

3.2 Hiệu năng (Performance Efficiency)

Hiệu năng của AutoGrade không đơn thuần là tốc độ, mà là kiểm soát thời gian chờ khi thực thi code và gọi dịch vụ AI (vốn có thể chậm/treo), cùng tính tất định của kết quả chấm.

NFR-PER-01 — Timeout thực thi trong sandbox

Phát biểu yêu cầu	Mỗi lần biên dịch và mỗi lần chạy testcase đều bị giới hạn thời gian; quá hạn thì tiến trình bị kill và testcase tính là timeout (TLE).
Lý do (Rationale)	Không có timeout, một bài nộp lỗi có thể treo cả tiến trình chấm. Giới hạn theo từng testcase giữ tổng thời gian chấm bị chặn trên.
Bằng chứng trong mã nguồn	<code>grading/sandbox.ts: setTimeout(...) → child.kill("SIGKILL"); trả timedOut = true</code> <code>grading/testcase.ts: status "timeout" khi r.timedOut</code> <code>config: runTimeoutMs=5000, compileTimeoutMs=15000 (cấu hình qua env)</code>
Use-case liên quan	UC Chấm điểm tự động (2.9); UC con Chạy testcase (sandbox).
Cách kiểm chứng	Testcase với code chạy quá 5s → đánh dấu TLE; tổng thời gian chấm không vượt (số testcase × timeout).
Ưu tiên	Bắt buộc

NFR-PER-02 — Timeout & retry có backoff khi gọi LLM

Phát biểu yêu cầu	Mọi lệnh gọi Gemini/Groq bị giới hạn 30s (AbortController); khi gặp lỗi tạm thời (429/500/502/503/504) hệ thống thử lại tối đa 3 lần với thời gian chờ tăng dần.
Lý do (Rationale)	Dịch vụ AI ngoài có thể chậm hoặc quá tải nhất thời. Timeout tránh treo pipeline; retry-with-backoff tăng khả năng thành công mà không dồn dập gây quá tải thêm.
Bằng chứng trong mã nguồn	<code>config.llmTimeoutMs = 30000; AbortController + setTimeout hủy request</code> <code>grading/style.ts & grading/aggregate.ts: retryable set {429,500,502,503,504}, tối đa 3 lần, chờ 700ms×attempt</code>
Use-case liên quan	UC Chấm điểm tự động (2.9): «include» Chấm phong cách, Tổng hợp điểm & nhận xét.
Cách kiểm chứng	Giả lập LLM trả 503 → hệ thống retry rồi rơi về fallback nếu vẫn lỗi, không treo quá 3×30s.
Ưu tiên	Bắt buộc

NFR-PER-03 — Chấm điểm tất định, tái lập được

Phát biểu yêu cầu	Điểm số cuối cùng được tính bằng công thức rubric tất định trên máy chủ; LLM chỉ viết nhận xét, không bao giờ quyết định điểm. Độ phức tạp được ước lượng bằng phân tích tĩnh thay vì đo thời gian chạy.
Lý do (Rationale)	Cùng một bài nộp phải cho cùng một điểm ở mọi lần chấm — điều kiện tiên quyết cho công bằng học vụ. Đo thời gian chạy bị nhiễu bởi tải máy/I-O nên không tái lập; phân tích tĩnh cho kết quả ổn định.
Bằng chứng trong mã nguồn	<code>grading/aggregate.ts: computeRubricScore() – điểm = (testcasePct·wTC + complexityPct·wCX)/(wTC+wCX)/10</code> <code>grading/complexity.ts: static structural analysis (độ sâu vòng lặp, sort, đệ quy) → lớp Big-O</code> <code>grading/style.ts: stylePct chỉ để nhận xét, KHÔNG vào điểm</code>
Use-case liên quan	UC Chấm điểm tự động (2.9); UC con Tổng hợp điểm & nhận xét, Phân tích độ phức tạp.

Cách kiểm chứng	Chấm cùng một bài nhiều lần → điểm không đổi; tắt khoá LLM → điểm vẫn giữ nguyên (chỉ đổi nhận xét).
Ưu tiên	Bắt buộc

NFR-PER-04 — Song song hoá & biên dịch một lần

Phát biểu yêu cầu	Các mô-đun độc lập (độ phức tạp và phong cách) chạy song song; code chỉ biên dịch một lần rồi tái sử dụng cho mọi testcase.
Lý do (Rationale)	Giảm tổng thời gian chấm mỗi bài mà không ảnh hưởng kết quả, tận dụng việc hai mô-đun không phụ thuộc nhau.
Bằng chứng trong mã nguồn	grading/pipeline.ts: <code>await Promise.all([measureComplexity(...), reviewStyle(...)])</code> grading/testcase.ts: <code>compileSource()</code> một lần → <code>runPrepared()</code> lặp cho từng testcase
Use-case liên quan	UC Chấm điểm tự động (2.9); UC con Chấm bài nộp.
Cách kiểm chứng	Đo thời gian: bài có N testcase chỉ biên dịch 1 lần; hai mô-đun nhận xét chạy chồng thời gian.
Ưu tiên	Nên có

3.3 Độ tin cậy (Reliability)

AutoGrade phụ thuộc vào dịch vụ AI ngoài và thực thi code dễ lỗi, nên độ tin cậy tập trung vào khả năng suy giảm mềm (graceful degradation) và quan sát được (observability): hệ thống vẫn hoạt động và cho điểm ngay cả khi thành phần phụ trợ hỏng.

NFR-REL-01 — Suy giảm mềm khi AI không khả dụng

Phát biểu yêu cầu	Nếu chưa cấu hình khoá Gemini/Groq hoặc dịch vụ AI lỗi, hệ thống dùng nhận xét dự phòng ghép từ số liệu tất định; điểm số vẫn được tính bình thường vì không phụ thuộc kết quả AI.
Lý do (Rationale)	Chấm điểm là chức năng cốt lõi và không được sập chỉ vì một dịch vụ ngoài trực trực. Tách bạch “điểm (tất định)” khỏi “nhận xét (AI)” khiến hệ thống bền vững.
Bằng chứng trong mã nguồn	grading/style.ts: <code>fallback()</code> khi <code>!config.gemini.apiKey</code> hoặc gọi lỗi grading/aggregate.ts: <code>fallbackProse()</code> khi <code>!config.groq.apiKey</code> hoặc lỗi Đặc tả UC 2.9 Alt flow 2: “điểm số vẫn được tính bình thường”
Use-case liên quan	UC Chấm điểm tự động (2.9) Alt flow “Chưa cấu hình API key”.
Cách kiểm chứng	Bỏ trống GEMINI_API_KEY & GROQ_API_KEY rồi chấm → vẫn ra điểm, nhận xét chuyển sang bản dự phòng.
Ưu tiên	Bắt buộc

NFR-REL-02 — Theo dõi tiến trình chấm (observability)

Phát biểu yêu cầu	Mỗi lần chấm được ghi một bản ghi <code>grading_jobs</code> với trạng thái <code>running/done/error</code> và thông điệp lỗi, thời điểm bắt đầu/kết thúc.
Lý do (Rationale)	Chấm hàng loạt có thể lỗi ở một vài bài; nhật ký job giúp truy vết bài nào lỗi vì sao mà không làm hỏng cả mẻ.

Bằng chứng trong mã nguồn	services/gradingService.ts: insert grading_jobs status "running" → update "done"/"error" kèm error_message db/schema.sql: bảng grading_jobs
Use-case liên quan	UC Chấm điểm tự động (2.9); UC Làm & nộp bài tự luyện (2.13).
Cách kiểm chứng	Chấm một mẻ có bài lỗi → job của bài đó ở trạng thái error với thông điệp; các bài khác vẫn done.
Ưu tiên	Nên có

NFR-REL-03 — Lưu file bền vững có dự phòng

Phát biểu yêu cầu	File upload được đẩy lên Supabase Storage (URL công khai bền vững); nếu Storage lỗi, hệ thống tự lưu tạm xuống đĩa local để ứng dụng vẫn chạy.
Lý do (Rationale)	Tránh cảnh “có tên file trong DB nhưng mất file trên đĩa” khi build lại; cơ chế fallback đảm bảo tính khả dụng khi Storage chưa sẵn sàng.
Bằng chứng trong mã nguồn	lib/upload.ts: saveFile() ưu tiên supabase.storage.upload → getPublicUrl; catch → ghi đĩa LEGACY_DIR lib/upload.ts: ensureBucket() idempotent, cache promise
Use-case liên quan	UC Nộp bài tập (2.8): «extend» Nộp kèm file; UC Tạo bài tập (đính kèm PDF).
Cách kiểm chứng	Upload khi Storage hoạt động → nhận URL https; giả lập Storage lỗi → file vẫn lưu và mở được qua /uploads.
Ưu tiên	Nên có

NFR-REL-04 — Health-check & không chặn khi chấm lỗi

Phát biểu yêu cầu	Có endpoint /api/health trả trạng thái sống; ở bài tự luyện, nếu chấm tự động lỗi thì bài nộp vẫn được lưu ở trạng thái chưa chấm, không chặn thao tác nộp.
Lý do (Rationale)	Health-check phục vụ giám sát/khởi động; tách việc “nộp” khỏi việc “chấm” đảm bảo sinh viên không mất bài chỉ vì grader tạm lỗi.
Bằng chứng trong mã nguồn	index.ts: app.get("/api/health", ... res.json({ ok: true }))) Đặc tả UC 2.13 Alt flow 1: “vẫn giữ bản nộp ở trạng thái chưa chấm, không chặn việc nộp bài”
Use-case liên quan	UC Làm & nộp bài tự luyện (2.13) Alt flow “Chấm tự động lỗi”.
Cách kiểm chứng	GET /api/health → {ok:true}; giả lập grader lỗi khi nộp bài tự luyện → bài vẫn được lưu.
Ưu tiên	Nên có

3.4 Tính khả chuyển (Portability & Compatibility)

Vì đây là đồ án nhiều thành viên với máy tính khác nhau, hệ thống phải chạy được trên nhiều hệ điều hành và hỗ trợ nhiều ngôn ngữ lập trình cho bài nộp.

NFR-POR-01 — Chạy đa nền tảng Windows/macOS/Linux

Phát biểu yêu cầu	Sandbox spawn trực tiếp trình biên dịch/thông dịch (không qua bash -c) để tránh lỗi WSL trên Windows; tự dò lệnh Python phù hợp từng OS và loại bỏ alias Python giả trên Windows.
-------------------	---

Lý do (Rationale)	Thành viên nhóm và máy chấm có thể dùng OS khác nhau. Việc phụ thuộc bash hay tên lệnh cứng sẽ vỡ trên Windows; cơ chế dò lệnh giúp “chạy được ngay” không cần cấu hình thủ công.
Bằng chứng trong mã nguồn	grading/sandbox.ts: IS_WIN; platformSpawn() bọc ulimit chỉ trên non-Windows grading/sandbox.ts: pythonCmd() thử python3/python/py; isRealPython() loại alias WindowsApps Java dùng -Xmx thay ulimit vì JVM cấp phát virtual memory lớn
Use-case liên quan	UC Chấm điểm tự động (2.9); UC con Chạy testcase (sandbox).
Cách kiểm chứng	Chạy pipeline trên Windows và Linux với cùng bài Python/C++/Java → cùng kết quả pass/fail.
Ưu tiên	Bắt buộc

NFR-POR-02 — Hỗ trợ đa ngôn ngữ lập trình

Phát biểu yêu cầu	Hệ thống biên dịch/chạy được Python, C++ và Java; tự suy ra ngôn ngữ theo thứ tự: khai báo → đuôi file → nội dung code. So khớp output có chuẩn hoá linh hoạt (bỏ ngoặc/dấu phẩy, so số theo giá trị).
Lý do (Rationale)	Bài tập lập trình dùng nhiều ngôn ngữ; auto-detect và chuẩn hoá output giảm sai lệch giả (ví dụ “[1, 2, 3]” của Python so với “1 2 3”).
Bằng chứng trong mã nguồn	grading/sandbox.ts: resolveLang(), compileSource() cho python/cpp/java grading/sandbox.ts: outputsEqual() so khớp chặt trước, rồi token linh hoạt
Use-case liên quan	UC Nộp bài tập (2.8); UC Làm & nộp bài tự luyện (2.13); UC Chạy thử code.
Cách kiểm chứng	Nộp cùng thuật toán bằng ba ngôn ngữ → đều được nhận diện và chấm đúng.
Ưu tiên	Bắt buộc

NFR-POR-03 — Cấu hình qua biến môi trường; độc lập hạ tầng

Phát biểu yêu cầu	Mọi tham số vận hành (cổng, URL, khoá, timeout sandbox/LLM, thư mục upload, bucket) đều đọc từ biến môi trường và có giá trị mặc định hợp lý; frontend/backend là hai project độc lập.
Lý do (Rationale)	Cho phép triển khai ở nhiều môi trường (DEV/PROD, Render/Vercel) mà không sửa code; tách hai project giúp mở rộng/triển khai riêng.
Bằng chứng trong mã nguồn	config.ts: toàn bộ tham số dùng process.env.X ?? default README.md: backend chạy độc lập, frontend gọi qua VITE_API_URL
Use-case liên quan	Xuyên suốt (thuộc tính triển khai).
Cách kiểm chứng	Đổi biến môi trường (ví dụ SANDBOX_TIMEOUT_MS, PORT) → hành vi thay đổi tương ứng, không cần sửa mã.
Ưu tiên	Nên có

3.5 Tính khả dụng (Usability)

Người dùng là sinh viên và giảng viên Việt Nam, nên trải nghiệm ưu tiên bản địa hoá tiếng Việt, phản hồi tức thời và thông báo lỗi dễ hiểu.

NFR-USA-01 — Bản địa hoá tiếng Việt

Phát biểu yêu cầu	Toàn bộ giao diện, thông báo lỗi và nhận xét AI đều bằng tiếng Việt; hệ thống xử lý đúng tên file tiếng Việt có dấu (giải mã UTF-8).
Lý do (Rationale)	Người dùng mục tiêu là sinh viên/giảng viên Việt Nam; tên file như “Thuyết trình.pdf” phải hiển thị đúng thay vì biến dạng.
Bằng chứng trong mã nguồn	Thông báo trong routes/*.ts và grading/*.ts đều tiếng Việt lib/upload.ts: origName() decode latin1 → utf8 cho tên file tiếng Việt Prompt LLM yêu cầu trả nhận xét tiếng Việt (style.ts, aggregate.ts)
Use-case liên quan	Xuyên suốt tất cả use-case.
Cách kiểm chứng	Upload file tên tiếng Việt → tên hiển thị đúng; các thông báo lỗi bằng tiếng Việt.
Ưu tiên	Bắt buộc

NFR-USA-02 — Chạy thử trước khi nộp

Phát biểu yêu cầu	Sinh viên có thể “Chạy thử” code với các testcase công khai để xem kết quả thực tế, không lưu và không tính điểm; testcase ẩn không bị lộ.
Lý do (Rationale)	Cho sinh viên vòng phản hồi nhanh kiểu LeetCode trước khi nộp chính thức, giảm nộp nhầm; vẫn giữ bí mật testcase ẩn để công bằng.
Bằng chứng trong mã nguồn	grading/testcase.ts: runSamples() chỉ chạy case client gửi lên, trả OUTPUT THỰC TẾ, không lưu Đặc tả UC con “Chạy thử code”: KHÔNG lưu, KHÔNG tính điểm
Use-case liên quan	UC Nộp bài tập (2.8) & UC Làm & nộp bài tự luyện (2.13): «extend» Chạy thử code.
Cách kiểm chứng	Bấm Chạy thử → thấy output từng case, không tạo bản nộp/điểm; testcase ẩn không xuất hiện.
Ưu tiên	Nên có

NFR-USA-03 — Thông báo lỗi rõ ràng, có hướng dẫn

Phát biểu yêu cầu	Lỗi trả về mã HTTP đúng ngữ nghĩa kèm thông điệp tiếng Việt cụ thể; lỗi thiếu công cụ (Python/g++/JDK) kèm hướng dẫn cài đặt.
Lý do (Rationale)	Thông báo mơ hồ khiến người dùng bế tắc. Ngữ nghĩa HTTP + hướng dẫn khắc phục giúp cả người dùng lẫn nhóm phát triển xử lý nhanh.
Bằng chứng trong mã nguồn	lib/http.ts: ApiError(status, message) + errorHandler tập trung grading/sandbox.ts: khi thiếu g++/python/javac → thông điệp kèm cách cài & thêm PATH
Use-case liên quan	Alternative flows của hầu hết use-case (dữ liệu không hợp lệ, thiếu quyền, quá hạn...).
Cách kiểm chứng	Gửi request thiếu trường → 400 kèm mô tả; chạy khi thiếu compiler → thông điệp hướng dẫn cài đặt.
Ưu tiên	Nên có

3.6 Khả bảo trì (Maintainability)

Đồ án cần dễ đọc, dễ mở rộng để nhóm cùng phát triển và để hoàn thiện dần các mô-đun chấm điểm.

NFR-MNT-01 — Kiểu tĩnh với TypeScript

Phát biểu yêu cầu	Cả frontend và backend viết bằng TypeScript với kiểu dữ liệu tường minh; có script typecheck riêng để kiểm tra kiểu mà không build.
Lý do (Rationale)	Kiểu tĩnh bắt lỗi sớm ở thời điểm biên dịch, giúp refactor an toàn và làm tài liệu sống cho cấu trúc dữ liệu (rubric, tiêu chí chấm...).
Bằng chứng trong mã nguồn	backend/package.json: script "typecheck": "tsc --noEmit"; devDep typescript ^5.7 grading/types.ts: định nghĩa GradeInput, RubricConfig, CriteriaResult, GradeResult frontend: tsc -b trong build
Use-case liên quan	Thuộc tính phát triển; nền cho toàn bộ pipeline.
Cách kiểm chứng	Chạy npm run typecheck → không lỗi kiểu; thay đổi cấu trúc rubric làm lộ ngay điểm gọi sai kiểu.
Ưu tiên	Bắt buộc

NFR-MNT-02 — Kiến trúc mô-đun & tách lớp

Phát biểu yêu cầu	Logic chấm điểm tách thành năm mô-đun độc lập (testcase, plagiarism, complexity, style, aggregate) do pipeline điều phối; tầng route (HTTP) tách khỏi tầng service (điều phối) và tầng grading (thuật toán).
Lý do (Rationale)	Tách bạch trách nhiệm cho phép thay/hoàn thiện từng mô-đun mà không đụng phần còn lại (ví dụ đối phương pháp đo độ phức tạp), và dễ kiểm thử từng phần.
Bằng chứng trong mã nguồn	src/grading/{testcase,plagiarism,complexity,style,aggregate}.ts + pipeline.ts điều phối src/routes/* (HTTP) → src/services/gradingService.ts (điều phối) → src/grading/* (thuật toán) README.md mục 6: mỗi file một tiêu chí
Use-case liên quan	UC Chấm điểm tự động (2.9) và các UC con «include» tương ứng từng mô-đun.
Cách kiểm chứng	Thay hiện thực một mô-đun (giữ nguyên interface) → pipeline vẫn chạy, các phần khác không đổi.
Ưu tiên	Bắt buộc

NFR-MNT-03 — Kiểm thử tự động pipeline

Phát biểu yêu cầu	Có script self-test cho pipeline chấm điểm để kiểm tra nhanh tính đúng đắn khi thay đổi.
Lý do (Rationale)	Cho phép kiểm tra hồi quy pipeline mà không cần dựng toàn bộ hệ thống, hỗ trợ phát triển an toàn.
Bằng chứng trong mã nguồn	backend/package.json: script "selftest": "tsx scripts/selftest-grading.ts"
Use-case liên quan	Thuộc tính phát triển cho UC Chấm điểm tự động (2.9).
Cách kiểm chứng	Chạy npm run selftest → pipeline chấm mẫu cho kết quả kỳ vọng.
Ưu tiên	Nên có

3.7 Khả mở rộng (Scalability) & giới hạn hiện tại

Phần này trình bày thẳng thắn năng lực mở rộng hiện tại và các giới hạn đã biết — điều thường được hỏi khi bảo vệ đồ án. Việc nêu rõ giới hạn là một điểm mạnh về mặt kỹ thuật, không phải điểm yếu của báo cáo.

NFR-SCL-01 — Backend stateless cho mở rộng ngang

Phát biểu yêu cầu	Backend không lưu trạng thái phiên trong bộ nhớ (xác thực bằng JWT, dữ liệu ở Supabase, file ở Storage), về nguyên tắc cho phép chạy nhiều bản sao sau load balancer.
Lý do (Rationale)	Stateless là điều kiện cần để mở rộng ngang; mọi trạng thái bền vững đã đẩy ra dịch vụ ngoài (DB/Storage).
Bằng chứng trong mã nguồn	middleware/auth.ts: xác thực bằng JWT, không session in-memory Trạng thái ở Supabase (DB) và Supabase Storage (file), không ở RAM tiến trình
Use-case liên quan	Thuộc tính triển khai.
Cách kiểm chứng	Khởi động hai tiến trình backend cùng trỏ một Supabase → cả hai phục vụ được cùng token/dữ liệu.
Ưu tiên	Nên có

NFR-SCL-02 — Giới hạn đã biết & hướng nâng cấp (Tương lai)

Bản hiện tại tối ưu cho quy mô lớp học/đồ án, chưa cho quy mô sản xuất lớn. Các giới hạn dưới đây được ghi nhận thẳng thắn kèm hướng khắc phục:

Giới hạn hiện tại	Ảnh hưởng	Hướng nâng cấp
Chấm hàng loạt bằng vòng lặp tuần tự trong tiến trình web (services/gradingService.ts).	Lớp rất đông có thể khiến một request chấm kéo dài; chấm chiếm CPU của tiến trình phục vụ.	Tách sang hàng đợi việc (job queue) + worker riêng, chấm bất đồng bộ, có thể chạy nhiều worker.
Sandbox “nhẹ”: tiến trình + timeout + ulimit, không cách ly hệ thống tệp/mạng triệt để.	Đủ cho môi trường tin cậy; chưa an toàn trước code cố ý phá hoại ở môi trường mở.	Chạy trong Docker/nsjail (chỉ cần đổi phần spawn trong platformSpawn), giới hạn mạng & filesystem.
CORS đang mở cho mọi origin (app.use(cors())).	Tiện cho phát triển; ở production kém an toàn.	Giới hạn origin whitelist theo domain frontend khi triển khai thật (đã ghi chú trong index.ts).
JWT_SECRET có giá trị mặc định “dev-secret-change-me”.	Nếu quên đổi khi triển khai, token có thể bị giả mạo.	Bắt buộc đặt JWT_SECRET ngẫu nhiên dài ở môi trường production; có thể thêm kiểm tra khởi động.

4. Ràng buộc & giả định

4.1 Ràng buộc kỹ thuật (Constraints)

- **Môi trường chạy:** Node.js ≥ 18; cần sẵn trình biên dịch/thông dịch tương ứng (Python 3, g++ hỗ trợ C++17, JDK 17+) trên máy chấm để chạy sandbox.
- **Cơ sở dữ liệu:** PostgreSQL trên Supabase; backend là nơi duy nhất chạm DB (không bật RLS vì không expose Supabase công khai ra frontend).
- **Dịch vụ AI:** Gemini và Groq là dịch vụ ngoài, cần khoá API; hệ thống thiết kế để vẫn chạy khi thiếu (NFR-REL-01).
- **Ngôn ngữ bài nộp:** Hỗ trợ Python, C++, Java ở bản hiện tại.

4.2 Giả định (Assumptions)

- Máy chấm là môi trường **tin cậy** (đồ án/nội bộ) — giả định này chi phối mức độ cách ly của sandbox (NFR-SEC-06, NFR-SCL-02).
- Quy mô sử dụng ở mức lớp học/đồ án, không phải hàng nghìn bài nộp đồng thời.

- Người dùng có quyền truy cập email đã đăng ký để nhận mã xác nhận/đặt lại mật khẩu.

4.3 Ma trận truy vết NFR ↔ Use-case

Bảng liên kết nhanh giữa các use-case chính và những NFR chi phối chúng, hỗ trợ đối chiếu khi bảo vệ và làm slide.

Use-case chính	NFR liên quan chính
Đăng ký / Xác nhận email / Đăng nhập / Quên mật khẩu	SEC-01, SEC-02, SEC-03, SEC-04, USA-03
Tạo lớp / Tham gia lớp / Tạo bài tập	SEC-02, SEC-05, MNT-02
Nộp bài tập (kèm chạy thử / kèm file)	SEC-06, SEC-08, USA-02, REL-03, POR-02
Chấm điểm tự động (pipeline 5 mô-đun)	SEC-06, SEC-07, PER-01, PER-02, PER-03, PER-04, REL-01, REL-02, POR-01, POR-02, MNT-02
Chỉnh sửa điểm / Công bố điểm	SEC-02, PER-03
Làm & nộp bài tự luyện	PER-03, REL-01, REL-04, USA-02
Xuất bảng điểm ra Excel	SEC-02, USA-01